

Guia de lectura - Sistema hibrido Dev Muscle

Para el usuario - Aprender haciendo - Cursor + Continue + Cline + Ollama

Generado: 2026-06-27 16:52

Empezar aquí

¿Qué tienes montado?

Un sistema híbrido de desarrollo en tu PC:

Cursor (tu IDE – no necesitas VS Code aparte)
├─ Continue → chat y autocomplete local
├─ Cline → agente que crea/edita archivos
├─ Dev Muscle → agente orquestador (@dev-muscle-orchestrator)
└─ Ollama → modelos de IA en tu máquina (gratis)

Importante: Cursor ya es un fork de VS Code. Las extensiones Continue y Cline viven dentro de Cursor. No hace falta abrir VS Code por separado.

Dónde está todo en tu disco

| Qué | Ruta |
|-----|
| Proyectos y workspace | `C:\Users\claud\dev-projects\hybrid-dev-system\workspace` |
| Documentación sistema | `...\hybrid-dev-system\docs` |
| Skills (agentes) | `C:\Users\claud\cursor\skills\dev-muscle-\*` |
| Config Continue | `C:\Users\claud\continue\config.yaml` |
| Config Cline | `C:\Users\claud\cline\data` |
| GitHub | <https://github.com/cscspachile-existe/hybrid-dev-system> |

Antes de cada sesión (30 segundos)

1. Ollama corriendo (icono en bandeja Windows)
2. Abrir **Cursor**
3. Abrir carpeta: `workspace\proyectos\tu-proyecto` (o crear uno)
4. Opcional: Settings → Usage → ver créditos restantes

Siguiente paso

Lee 02-como-funciona-el-sistema.md y luego haz 03-practica-ejercicio-1.md.

Cómo funciona el sistema (para humanos)

La idea en una frase

Local primero (gratis), Cursor cloud solo si hace falta, memoria en archivos para no repetir.

Las piezas y qué hace cada una

1. Cursor — tu mesa de trabajo

| Rol | Qué es |
|-----|
| IDE | Editor de código, archivos, terminal integrada |
| Tab | Autocompletado rápido (usa créditos del plan) |
| Agent cloud | Agente potente Opus/Sonnet (usa créditos) |
| Rules / Skills | Memoria persistente del proyecto |

Cuándo usarlo: siempre abierto. Agent cloud solo para tareas difíciles.

2. Ollama — el motor local

| Rol | Qué es |
|-----|
| Servidor local | Corre modelos en tu PC |
| Sin internet | Inferencia sin API |
| Sin créditos | No gasta plan de Cursor |

Modelos que tienes: qwen-coder-32k, qwen2.5-coder:14b, 1.5b, nomic-embed-text

Cuándo usarlo: siempre detrás de Continue y Cline (automático).

3. Continue — asistente al lado

| Rol | Panel en Cursor |
|-----|
| Chat | Preguntas, explicaciones |
| Autocomplete | Sugerencias mientras escribes (local) |
| @Codebase | Buscar en todo el proyecto |
| Edit | Modificar código seleccionado |

Cuándo usarlo:

- "¿Qué hace este archivo?"
- "Explícame este error"
- Autocomplete sin gastar Tab de Cursor
- `@Codebase ¿dónde está la validación?`

Costo: 0 créditos

4. Cline — el ejecutor

| Rol | Panel en Cursor |
|-----|
| Plan | Propone plan sin tocar archivos |
| Act | Crea/edita archivos, corre comandos |
| Aprobación | Tú apruebas cada paso |

Cuándo usarlo:

- Crear archivos nuevos
- Refactor en varios archivos
- Instalar dependencias
- CRUD, componentes, scripts

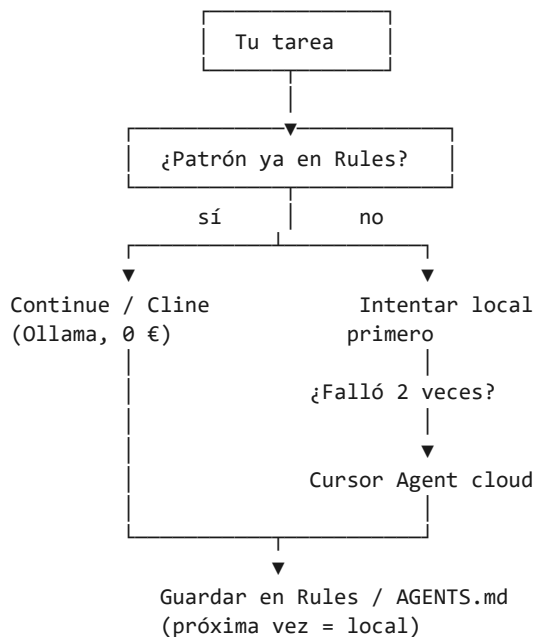
Costo: 0 créditos (con Ollama)

5. Dev Muscle — tu orquestador

| Rol | Cursor Chat con `@dev-muscle-orchestrator` |
|-----|
| Decide | Local vs cloud |
| Delega | Frontend, backend, WordPress |
| Recuerda | Lee AGENTS.md y Rules |

Cuándo usarlo: cuando no sabes qué herramienta elegir.

Diagrama del flujo



VS Code vs Cursor — aclaración

Pregunta	Respuesta
¿Necesito VS Code?	****No****
¿Continue/Cline funcionan?	****Sí, dentro de Cursor****
¿Es el mismo esquema que "VS Code + Continue + Cline + Ollama"?	****Sí****, pero el editor es ****Cursor****, no VS Code aparte

Siguiente

Practica con 03-practica-ejercicio-1.md

Práctica — Ejercicio 1 (aprender haciendo)

Tiempo: ~20 minutos. Objetivo: completar una tarea real usando solo herramientas locales (0 créditos).

Paso 1: Crear proyecto de práctica

```
cd C:\Users\claud\dev-projects\hybrid-dev-system\workspace\proyectos
mkdir practica-01
cd practica-01
git init
Copy-Item ..\..\..\templates\AGENTS.md .
Copy-Item -Recurse ..\..\..\templates\cursor .
```

Abre en Cursor: File → Open Folder → practica-01

Paso 2: Continue — preguntar (2 min)

1. Abre panel **Continue** (barra lateral)
2. Escribe:

@AGENTS.md Resume en 3 bullets qué stack usaré en este proyecto de práctica

3. Observa: responde con Ollama local, sin créditos.

Paso 3: Cline — crear código (10 min)

1. Abre panel **Cline**
2. Verifica: Provider **Ollama**, Model **qwen-coder-32k**
3. Modo **Plan**, escribe:

Crea una página HTML simple "Hola Dev Muscle" con:

- Título h1
- Párrafo de bienvenida en español
- Estilos CSS inline, fondo oscuro
- Guardar como index.html en la raíz del proyecto

4. Revisa el plan → **Act** → aprueba cambios
5. Abre `index.html` en el navegador (clic derecho → Open with Live Server o abrir archivo)

Aprendiste: Cline crea archivos de forma autónoma con aprobación.

Paso 4: Continue — editar (3 min)

1. Selecciona el `<h1>` en `index.html`
2. En Continue, usa **Edit** (o Ctrl+L con selección):

Cambia el título a "Práctica 01 – Sistema Híbrido" y añade un botón "Empezar"

Aprendiste: Continue edita código existente.

Paso 5: Dev Muscle — orquestar (3 min)

1. Cursor Chat (`Ctrl+L`):

@dev-muscle-orchestrator

Acabo de crear index.html con Cline.

¿Debo guardar algún patrón en Rules para futuras páginas HTML de práctica?

2. Si propone una Rule, créala en `.cursor/rules/html-practica.mdc`

Aprendiste: el orquestador guía memoria y convenciones.

Paso 6: Git — respaldar (2 min)

```
git add .
git commit -m "practica-01: primera pagina con Cline local"
```

Checklist — ¿lo lograste?

- [] Continue respondió en chat
- [] Cline creó `index.html` en Plan → Act
- [] Continue editó el archivo
- [] Dev Muscle sugirió o creó una Rule

- [] Commit en git

Siguiente ejercicio

Cuando domines esto, prueba 04-prompts-y-navegacion.md con una tarea más grande (componente React o endpoint API).

Prompts y navegación entre herramientas

Regla de oro

> Empieza siempre local. Escala a Cursor cloud solo si local falla 2 veces o la tarea es claramente difícil.

Tabla: qué herramienta usar

Quiero... Herramienta Panel / atajo
----- ----- -----
Preguntar sobre código Continue Continue
Completar mientras escribo Continue Editor
Buscar en todo el repo Continue `@Codebase` en chat
Crear archivos / refactor Cline Cline → Plan → Act
No sé por dónde empezar Dev Muscle Cursor Chat `@dev-muscle-orchestrator`
Bug muy difícil Cursor Agent `Ctrl+I`
Ver créditos Cursor Settings → Usage

Prompts listos para copiar

Continue — explicar

@src/archivo.ts Explica qué hace este archivo en español, en 5 bullets

Continue — buscar

@Codebase ¿dónde se valida el email del usuario?

Cline — crear feature

Crea [descripción].
Sigue AGENTS.md y .cursor/rules/.
Stack: [tu stack].
Responde/ejecuta en español en comentarios de commit.

Dev Muscle — orquestador

@dev-muscle-orchestrator

Tarea: [describe aquí]
Prioridad: local si es posible
Stack: ver AGENTS.md

Dev Muscle + especialista frontend

@dev-muscle-orchestrator @dev-muscle-frontend

Crea componente Button con variantes primary/secondary en Tailwind.
Local primero.

Escalar a cloud (solo cuando haga falta)

@dev-muscle-orchestrator

Escalar a Cursor cloud. Créditos OK.
Tarea: [descripción compleja]
Incluye @AGENTS.md

Cómo moverte entre paneles en Cursor

Barra lateral izquierda:

— Explorer	→ archivos
— Search	→ buscar texto
— Continue	→ chat local + autocomplete
— Cline	→ agente Plan/Act
— (otros)	

Atajos útiles:

Ctrl+L → Cursor Chat (Agent / Dev Muscle)
Ctrl+I → Cursor Agent (cloud)
Ctrl+` → Terminal
Ctrl+Shift+P → Command Palette

Flujo típico de una sesión

1. Abrir proyecto en Cursor
 2. Settings → Usage (créditos)
 3. Tarea nueva:
 - a. ¿Solo pregunta? → Continue
 - b. ¿Crear código? → Cline Plan → Act
 - c. ¿Duda de herramienta? → @dev-muscle-orchestrator
 4. Al terminar algo nuevo → Rule o AGENTS.md
 5. git commit
-

Errores comunes

Error	Solución
Continue no responde	`ollama serve` o reiniciar Ollama
Cline "connection refused"	Ollama no corre
Cline muy lento	Compact prompt ON (ya configurado)
Gasto créditos sin querer	Auto OFF, usar Continue/Cline primero
IA repite lo mismo	Crear Rule en `.cursor/rules/`

Práctica sugerida

Repite 03-practica-ejercicio-1.md cambiando la tarea:

- Ejercicio 2: Cline crea `styles.css` separado
 - Ejercicio 3: Continue `@Codebase` en un proyecto más grande
 - Ejercicio 4: Dev Muscle te pide escalar a cloud en un bug inventado
-

Cuándo escalar a Cursor cloud

Antes de gastar créditos

1. ****Settings → Usage**** — ¿cuánto queda del mes?
 2. ****¿Existe Rule?**** — busca en ``.cursor/rules/`` y ``AGENTS.md``
 3. ****¿Probaste local 2 veces?**** — Cline o Continue
-

Usa cloud SOLO si

Situación	Ejemplo
Cline falló 2× misma tarea	Bug que local no entiende
Arquitectura nueva	Diseñar sistema desde cero
Seguridad crítica	Auth, permisos, auditoría
Integración desconocida	API nunca usada en el proyecto
Tú lo pides explícitamente	"Necesito Opus para esto"

Config recomendada al escalar

Cursor Agent (Ctrl+I):

- Auto: OFF
- MAX Mode: OFF
- Modelo: Composer 2.5 Fast (normal) o Sonnet (difícil)

Incluye en el prompt:

```
@AGENTS.md
@.cursor/rules/hybrid-workflow.mdc
```

[Tu tarea]

Después de resolver con cloud (obligatorio)

Para que no vuelvas a pagar la misma tarea:

1. Crea ``.cursor/rules/nombre-patron.mdc`` con lo aprendido
2. Actualiza ``AGENTS.md`` si hubo decisión de arquitectura
3. ``git commit -m "docs: add pattern X"``

La próxima vez → Cline/Continue local.

Ejemplo

❌ Mal: Cada CRUD → Cursor Agent (gasta créditos)

✅ Bien:

1. Primer CRUD usuarios → Cursor cloud (difícil)
 2. Guardar patrón en `rules/api-crud.mdc`
 3. CRUD productos, pedidos... → Cline local (gratis)
-

Memoria y organización

Dos tipos de documentación

| Para | Carpeta | Ejemplo |

|-----|-----|-----|
| **Ti** (leer, aprender) | `workspace/docs-usuario/` | Este manual |
| **Agentes** (reglas, contexto) | `workspace/docs-agentes/` + `.cursor/rules/` | Patrones API |

Memoria del sistema híbrido

La IA no aprende sola. Tú (o el agente) escribes archivos:

Artefacto	Ubicación	Quién lo lee
AGENTS.md	Raíz del proyecto	Todos
Rules	`.cursor/rules/\*.mdc`	Cursor, Cline
Skills	`~/cursor/skills/`	Cursor Agent
Cline rules	`.clinerules/`	Cline
Git history	Commits	@Codebase

Ritual al terminar sesión (2 min)

- ☐ ¿Resolví algo nuevo con cloud?
→ Crear/actualizar Rule
 - ☐ ¿Decisión de arquitectura?
→ Actualizar AGENTS.md
 - ☐ ¿Screenshot útil?
→ workspace/imagenes/capturas/
 - ☐ ¿Script reutilizable?
→ workspace/scripts/
 - ☐ git add + commit + push
-

Respaldar en GitHub

Repo: <https://github.com/cscspachile-existe/hybrid-dev-system>

```
cd C:\Users\claud\dev-projects\hybrid-dev-system
git add .
git commit -m "docs: actualizar workspace y practica"
git push
```

No subas: .db grandes, .env, passwords, ejecutables pesados.

Estructura workspace (resumen)

```
workspace/
├── proyectos/      ← tu código
├── scripts/        ← automatización
├── base-datos/     ← schemas (no dumps sensibles)
├── imagenes/       ← capturas, diagramas
├── docs-usuario/   ← manuales (tú)
├── docs-agentes/   ← contexto IA
├── referencia/     ← cheatsheets, links
└── ejecutables/   ← solo notas (INSTALADOS.md)
```

Aprender haciendo — ruta sugerida

| Semana | Actividad |

|-----|-----|

| 1 | Ejercicio 1 + Continue chat |

| 2 | Cline Plan/Act en proyecto real |

| 3 | Crear 3 Rules de patrones tuyos |

| 4 | Escalar 1 vez a cloud + documentar patrón |

| 5 | Proyecto completo solo local |
